DEEP TRANSFER LEARNING FOR MALIGNANCY DETECTION IN OVARIAN ULTRASOUND IMAGES USING MOBILENETV3

A PROJECT REPORT

Submitted By

PARUPUDI SRIDATTA BHARADWAJ

[Reg. No. CH.SC.U4CSE24234]

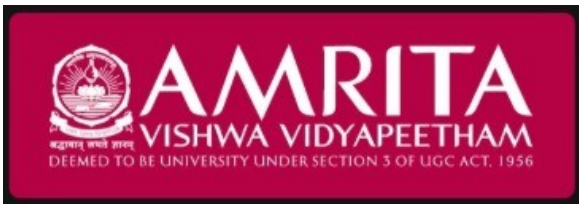
in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

Dr. Nirmalraj J

Submitted to

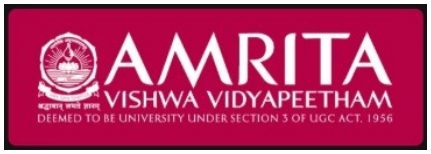


AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF COMPUTING

CHENNAI – 601103

October 2026



SCHOOL OF
COMPUTING
CHENNAI

DECLARATION BY THE CANDIDATE

I declare that the report entitled **DEEP TRANSFER LEARNING FOR MALIGNANCY DETECTION IN OVARIAN ULTRASOUND IMAGES USING MOBILENETV3** submitted by me for the degree of Bachelor of Technology is the record of the project work carried out by me under the guidance of **Dr. Nirmalraj J** and this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, titled in this or any other University or other similar institution of higher learning.

(Signature)
PARUPUDI SRIDATTA BHARADWAJ
[Reg. No. CH.SC.U4CSE24234]

ABSTRACT

Ovarian cancer is among the deadliest gynaecological cancers, largely because it is often found late, and transvaginal ultrasound is the first imaging test used to decide whether an adnexal mass is benign or malignant. This project develops and rigorously evaluates a deep-learning baseline, named **E0**, that classifies a single two-dimensional B-mode ovarian ultrasound image as benign or malignant.

The work uses the public OTU_2D dataset (1,469 images in eight tumour categories), in which only high-grade serous carcinoma is malignant. Preprocessing removes on-screen annotations, crops and letterboxes every frame to 224 × 224 pixels and removes near-duplicate frames, leaving 1,343 images of which only 50 (3.7%) are malignant. The data is divided into five grouped, stratified cross-validation folds so that near-identical frames never appear on both sides of a split.

The E0 model is an ImageNet-pretrained MobileNetV3-Large network with a global-average-pooling, dropout and single sigmoid output head, trained with inverse-frequency class weights and light data augmentation. Training runs in two stages: 30 epochs on the classification head with the backbone frozen, followed by 90 epochs of fine-tuning on the upper half of the backbone. For every fold the checkpoint with the best validation AUC is kept and the decision threshold is chosen on the validation fold using Youden's J statistic, so the test fold is never used for any decision.

Pooled over all five folds, E0 reached an out-of-fold ROC-AUC of **0.819** (fold mean 0.841 ± 0.056) and detected **34 of the 50** malignant images (sensitivity 68%, 95% CI 54–79%) at a specificity of **87.9%**. An independent earlier run of the same configuration gave 35/50 and a pooled AUC of 0.823, confirming that the pipeline is reproducible. Learning curves and Grad-CAM analysis point to the small number of malignant examples as the main limitation. The baseline provides a reproducible reference point for future work on lighter and more robust models.

Keywords: Deep Learning, Transfer Learning, Ovarian Tumour Classification, Ultrasound Imaging, MobileNetV3, Class Imbalance, Grouped Cross-Validation, Grad-CAM, Medical Image Analysis

ACKNOWLEDGEMENT

This project work would not have been possible without the contribution of many people. It gives me immense pleasure to express my profound gratitude to our honourable Chancellor for her blessings and for being a constant source of inspiration. I am indebted to the Director of Amrita School of Computing for providing all the facilities and support that made a valuable learning experience possible.

I register my special thanks to the Principal, the Chairperson and the Program Chair of the Department of Computer Science and Engineering for the support given to me in the successful conduct of this project. I wish to express my sincere gratitude to my project guide for the inspiring guidance, personal involvement and constant encouragement throughout the course of this work.

I am grateful to the Project Coordinator, the Review Panel Members and the entire faculty of the Department of Computer Science and Engineering for their constructive criticism and valuable suggestions, which have been a rich source of improvement for this work. Finally, I thank my family and friends for their patience and encouragement.

PARUPUDI SRIDATTA BHARADWAJ
[Reg. No. CH.SC.U4CSE24234]

TABLE OF CONTENTS

LIST OF TABLES ..... **vii**
LIST OF FIGURES ..... **viii**
LIST OF SYMBOLS AND ABBREVIATIONS ..... **ix**
I. INTRODUCTION **1**
1.1. Overview 2
1.2. Motivation 3
1.3. Problem Statement 3
1.4. Objectives 4
1.5. Scope of the Project 4
II. LITERATURE REVIEW AND PROBLEM BACKGROUND **6**
2.1. Ovarian Tumours and Ultrasound Assessment 6
2.2. Deep Learning for Ovarian Ultrasound 6
2.3. Transfer Learning and Efficient CNN Architectures 7
2.4. Learning from Imbalanced Medical Data 8
2.5. Explainability and Shortcut Learning 8
2.6. Limitations of Existing Approaches 8
III. SYSTEM ANALYSIS AND METHODOLOGY **10**
3.1. System Architecture 10
3.2. Technology Stack 11
3.2.1. Python Environment 11
3.2.2. TensorFlow and Keras 12
3.2.3. Image Processing Libraries 12
3.2.4. Evaluation and Visualisation Libraries 12
3.3. Dataset Description 12
3.4. Model Selection 14
3.5. Methodology Overview 15
IV. SYSTEM DESIGN AND IMPLEMENTATION **16**
4.1. Overall System Design 16
4.2. Data Cleaning and Preprocessing 16
4.3. Leakage-aware Cross-validation Design 17
4.4. Input Pipeline and Data Augmentation 18
4.5. Handling Class Imbalance 19
4.6. Network Architecture 19
4.7. Two-stage Training Strategy 20
4.8. Checkpoint Selection and Decision Threshold 21

4.9. Evaluation Metrics.....21
4.10. Implementation and Software Organisation.....22
V. RESULTS AND DISCUSSION.....23
5.1. Testing Environment.....23
5.2. Test Cases and Evaluation Set-up.....23
5.3. Training Behaviour.....24
5.4. Per-fold Performance.....25
5.5. Pooled Cross-validation Performance.....26
5.6. Reproducibility of the Baseline.....28
5.7. Final Model.....29
5.8. Model Interpretability with Grad-CAM.....30
5.9. Discussion.....31
VI. CONCLUSIONS AND SCOPE FOR FURTHER WORK.....32
6.1. Conclusions.....32
6.2. Limitations.....32
6.3. Future Enhancements.....33
VII. REFERENCES.....35

LIST OF TABLES

Table 1 — Technology Stack Summary.....11
Table 2 — Summary of the Dataset Used.....13
Table 3 — Comparison of Candidate Backbone Networks (ImageNet, 224 × 224 input).....14
Table 4 — Cross-validation Fold Composition.....18
Table 5 — E0 Training Configuration.....20
Table 6 — Testing Environment.....23
Table 7 — Pipeline Test Cases.....23
Table 8 — Per-fold Test Results of E0.....25
Table 9 — Pooled 5-fold Out-of-fold Results of E0.....26
Table 10 — Reproducibility of E0 Across Two Independent Runs.....28
Table 11 — Final E0 Model Summary.....29

LIST OF FIGURES

Fig. 1. System architecture of the E0 ovarian ultrasound classification pipeline.....10
Fig. 2. One cleaned 224 × 224 example from each of the eight categories.....13
Fig. 3. Class distribution of the cleaned dataset.....14
Fig. 4. Methodology overview of the E0 training and evaluation procedure.....15
Fig. 5. Raw OTU_2D frames (top) and the corresponding cleaned 224 × 224 images (bottom).....17
Fig. 6. Rotation of training, validation and test folds across the five cross-validation runs.....18
Fig. 7. E0 network architecture: MobileNetV3-Large backbone with a pooled, regularised sigmoid head.....20
Fig. 8. Loss and ROC-AUC curves for fold 0 (training, validation and test).....24
Fig. 9. Training and validation ROC-AUC for all five folds.....25
Fig. 10. ROC-AUC, sensitivity and specificity on each test fold.....26
Fig. 11. Pooled and per-fold ROC curves with the operating point at the Youden thresholds.....27
Fig. 12. Pooled confusion matrix over all 1,343 out-of-fold predictions.....28
Fig. 13. Loss and ROC-AUC curves of the final model trained on folds 0–3.....29
Fig. 14. Grad-CAM heat maps for E0 out-of-fold predictions (green outline = annotated lesion). Rows: true positives, false negatives, false positives, true negatives 30

LIST OF SYMBOLS AND ABBREVIATIONS

AUC	Area Under the Curve
BCE	Binary Cross-Entropy
CI	Confidence Interval
CNN	Convolutional Neural Network
CPU / GPU	Central / Graphics Processing Unit
CV	Cross-Validation
FN / FP	False Negative / False Positive
Grad-CAM	Gradient-weighted Class Activation Mapping
HGSC	High-Grade Serous Carcinoma
IOTA	International Ovarian Tumour Analysis
J	Youden's J statistic (sensitivity + specificity − 1)
lr	Learning rate
MMOTU	Multi-Modality Ovarian Tumour Ultrasound dataset
NPV	Negative Predictive Value
O-RADS	Ovarian-Adnexal Reporting and Data System
OTU_2D	Two-dimensional ovarian tumour ultrasound subset of MMOTU
pHash	Perceptual Hash
PPV	Positive Predictive Value (precision)
PR-AUC	Area under the Precision–Recall curve
ROC	Receiver Operating Characteristic
SE	Squeeze-and-Excitation
TN / TP	True Negative / True Positive
WSL	Windows Subsystem for Linux

I. INTRODUCTION

Ovarian cancer causes more deaths than any other gynaecological cancer, and the main reason is timing: most women are diagnosed only after the disease has spread beyond the ovary, when the chance of cure is much lower [1]. When a woman presents with an ovarian or adnexal mass, the first and most widely used imaging test is transvaginal ultrasound. On the basis of that scan, a clinician has to decide whether the mass is almost certainly benign and can be watched, or whether it looks suspicious and should be referred to a gynaecological oncologist for surgery.

This decision is harder than it sounds. Benign lesions such as endometriomas (chocolate cysts), teratomas and cystadenomas are very common, and several of them can share features with cancer, such as solid parts, thick septa or irregular walls. Structured scoring systems such as the IOTA simple rules [2] and the O-RADS ultrasound risk classification [3] were developed to make the assessment more consistent, yet the final reading still depends strongly on the skill and experience of the examiner. Expert sonographers are scarce, especially outside large hospitals.

Deep learning has changed what is possible in medical image analysis. Convolutional neural networks (CNNs) trained on large collections of labelled images now match specialists on several narrow diagnostic tasks [7], [8]. An automated system that looks at an ovarian ultrasound image and returns a malignancy score could therefore act as a second reader, helping less experienced examiners decide which patients need urgent referral.

This project builds and carefully evaluates such a system using the public OTU_2D dataset [4]. The model, called **E0 (baseline)**, is a MobileNetV3-Large network [14] pretrained on ImageNet [10] and fine-tuned to separate high-grade serous carcinoma from seven benign categories. E0 is deliberately simple. It uses grayscale images, light augmentation, class weighting and a fixed two-stage training schedule, so that it can serve as a trustworthy reference point. Most of the effort went into making its performance estimate honest: removing duplicate frames, preventing leakage between training and test images, choosing every setting on validation data only, and reporting results with confidence intervals.

The main contributions of this project are:

- **A cleaned, leakage-aware dataset pipeline** that removes screen overlays, de-duplicates near-identical frames with perceptual hashing, and builds five grouped, stratified cross-validation folds in which visually near-identical frames never straddle a split.
- **A reproducible transfer-learning baseline (E0)** based on MobileNetV3-Large with a two-stage training schedule, inverse-frequency class weights and a validation-chosen decision threshold.
- **An honest evaluation protocol** in which each of the 50 malignant images is tested exactly once by a model that never saw it, with pooled metrics, Wilson confidence intervals, a reproducibility check and Grad-CAM explanations.

The report is organised as follows. Chapter II reviews related work on ovarian ultrasound assessment, deep learning for medical imaging, efficient CNN architectures and evaluation under class imbalance. Chapter III describes the system architecture, technology stack, dataset and model selection. Chapter IV presents the design and implementation of each stage of the pipeline. Chapter V reports and discusses the experimental results. Chapter VI concludes the report, lists the limitations and outlines future work, including the training of a lighter model.

1.1. Overview

The project addresses a binary image-classification problem: given one two-dimensional B-mode ultrasound image of an ovary, estimate the probability that the lesion it shows is malignant. The data comes from OTU_2D, the two-dimensional part of the Multi-Modality Ovarian Tumour Ultrasound (MMOTU) collection [4], which contains 1,469 images from eight diagnostic categories. Only one category, high-grade serous carcinoma, is malignant; the other seven are benign lesions or normal ovaries.

The system is organised as a pipeline of independent stages. Raw frames are first cleaned of on-screen annotations, cropped to the ultrasound fan and resized to a fixed square input. Near-duplicate frames are removed and similar frames are grouped. The data is then split into five folds for cross-validation. For each fold a MobileNetV3-Large network is trained in two stages, the best checkpoint is selected on a validation fold, a decision threshold is fixed on the same validation fold, and the model is finally evaluated on a test fold it has never seen. The predictions from the five test folds are

pooled to give one overall estimate of performance. A final model trained on four folds is also produced for later use.

1.2. Motivation

The motivation for this work is the gap between how common adnexal masses are and how few expert readers are available. A large share of ovarian masses found on ultrasound are benign, but a missed cancer has very serious consequences, while over-referral of benign masses leads to unnecessary surgery and anxiety. A tool that gives a reliable, consistent second opinion would help non-expert examiners triage patients more safely.

A second motivation is methodological. Many published studies of deep learning on medical images report very high accuracy on small datasets, but these numbers are often inflated by near-duplicate images appearing in both training and test sets, by thresholds tuned on the test data, or by reporting accuracy on heavily imbalanced data where predicting "benign" for everything already scores above 95%. This project therefore puts as much weight on *how* the model is evaluated as on the model itself, so that the reported numbers can be trusted and future improvements can be measured against a solid baseline.

1.3. Problem Statement

The task is to build a model that classifies a single ovarian ultrasound image as benign or malignant, and to estimate its real performance honestly. Several properties of the available data make this difficult.

First, the data is **severely imbalanced**. After cleaning, only 50 of the 1,343 images (3.7%) are malignant, about one malignant image for every 26 benign ones. A naive model can reach high accuracy by ignoring the malignant class completely. Second, the dataset contains **near-duplicate frames** and carries **no patient identifiers**, so a random split could place two almost identical frames of the same lesion in the training and the test set and make the model look better than it is. Third, the images contain **on-screen artefacts** such as calipers, measurement tables and machine text, which a network may learn as shortcuts instead of the anatomy. Finally, the dataset is **small**, so any single train/test split gives a very noisy estimate.

The problem, therefore, is not only to train an accurate classifier but to design a complete, reproducible pipeline that cleans the data, prevents information leakage, handles extreme class imbalance, and reports sensitivity and specificity with honest uncertainty.

1.4. Objectives

The specific objectives of this project are:

- To build a preprocessing pipeline that removes on-screen overlays, isolates the ultrasound fan, standardises the image size and removes near-duplicate frames.
- To design a grouped, stratified five-fold cross-validation scheme that keeps visually near-identical frames in the same fold and spreads the 50 malignant images evenly across folds.
- To fine-tune an ImageNet-pretrained MobileNetV3-Large network for binary malignancy classification, using class weights and data augmentation to deal with the class imbalance.
- To select checkpoints and decision thresholds using validation data only, so that the test folds remain untouched until the final evaluation.
- To evaluate the model with ROC-AUC, PR-AUC, sensitivity, specificity and predictive values, with 95% confidence intervals, and to verify that the result is reproducible.
- To inspect what the network has learned using Grad-CAM, and to identify the main limitations that future work must address.

1.5. Scope of the Project

The scope of this project is the design, implementation and evaluation of a single baseline classifier, E0, as a software system. The work covers data cleaning, dataset splitting, model training, threshold selection, evaluation and visual explanation. All experiments use the public OTU_2D dataset; no new patient data was collected and no integration with hospital information systems was attempted.

The model works on single still images and makes a binary decision (high-grade serous carcinoma versus everything else). Multi-class diagnosis of the eight lesion types, video sequences, three-dimensional or Doppler-specific analysis, and

prospective clinical validation are outside the scope of this project. The results are intended as a proof of concept and a reference baseline, not as a validated clinical tool. Training a lighter model for faster inference is identified as future work in Chapter VI.

II. LITERATURE REVIEW AND PROBLEM BACKGROUND

Research relevant to this project falls into several connected areas. Clinical work on ovarian ultrasound has produced structured rules for separating benign from malignant masses. Deep-learning research has shown that CNNs can learn this distinction directly from images, provided that enough labelled data is available. Work on transfer learning and efficient architectures explains how good models can be trained on small medical datasets with modest hardware. Finally, a growing body of work on evaluation and explainability warns about the ways in which reported results can be misleading. This chapter reviews each area and closes with the gaps that motivate the present work.

2.1. Ovarian Tumours and Ultrasound Assessment

Ovarian cancer is a leading cause of death from gynaecological cancer worldwide [1]. High-grade serous carcinoma is the most common and most aggressive subtype, and it is the malignant class in the dataset used here. Because symptoms are vague, ultrasound assessment of an incidentally found or symptomatic adnexal mass is often the point at which cancer is first suspected.

To reduce the dependence on examiner experience, the International Ovarian Tumour Analysis (IOTA) group proposed the “simple rules” [2], a set of benign and malignant ultrasound features (for example, a unilocular cyst versus an irregular solid tumour with strong blood flow). The rules classify many masses reliably but leave a proportion of cases inconclusive. More recently, the O-RADS ultrasound system [3] introduced a standard lexicon and risk categories that link ultrasound appearance to the likelihood of malignancy and to recommended management. Both systems remain manual: the examiner must recognise each feature on the image. This creates a natural opportunity for automated image analysis to support the examiner.

2.2. Deep Learning for Ovarian Ultrasound

Several studies have applied deep learning directly to ovarian ultrasound images. Christiansen et al. [5] trained an ensemble of deep neural networks on ultrasound images of ovarian tumours and reported diagnostic performance comparable to expert subjective assessment, suggesting that the visual cues used by experts can be

learned from data. Gao et al. [6] carried out a large retrospective multicentre study in China and showed that a deep convolutional network trained on pelvic ultrasound images could identify ovarian cancer accurately and could also improve the performance of radiologists who used it as an aid.

These studies relied on large private datasets. Public data is far more limited. The MMOTU dataset released by Zhao et al. [4] is one of the few publicly available ovarian ultrasound collections; it provides two-dimensional (OTU_2D) and contrast-enhanced (OTU_CEUS) images with eight category labels and segmentation masks, and was introduced mainly as a benchmark for segmentation and domain adaptation. Its two-dimensional subset is the dataset used in this project. Because it contains only 53 raw images of the malignant category and no patient identifiers, results obtained on it must be interpreted with great care, which is a central theme of this report.

2.3. Transfer Learning and Efficient CNN Architectures

When labelled medical images are scarce, the standard approach is transfer learning: a network is first trained on a large natural-image dataset such as ImageNet [10] and then fine-tuned on the medical task. Litjens et al. [7] surveyed deep learning in medical image analysis and found that pretrained CNNs had become the dominant approach across modalities. Esteva et al. [8] showed that a pretrained CNN fine-tuned on skin-lesion photographs could reach dermatologist-level classification. Raghu et al. [9] examined transfer learning for medical imaging more closely and found that much of its benefit comes from better-conditioned initial weights and faster convergence, and that smaller, lighter architectures can perform close to large ones on medical tasks.

Architectures designed for efficiency are therefore attractive. Very deep networks such as ResNet [11] achieve high accuracy but carry tens of millions of parameters. The MobileNet family was designed to reduce computation: MobileNetV1 [12] introduced depthwise-separable convolutions, MobileNetV2 [13] added inverted residual blocks with linear bottlenecks, and MobileNetV3 [14] combined these with squeeze-and-excitation attention [29], the hard-swish activation and an architecture found by neural architecture search. EfficientNet [15] scaled network depth, width and resolution together. MobileNetV3-Large offers ImageNet accuracy similar to much larger networks with only about 5.4 million parameters, which makes it a good backbone for a small dataset where a very large model would overfit easily.

2.4. Learning from Imbalanced Medical Data

In screening-type problems the condition of interest is rare, and the data reflects this. With only a few per cent positive examples, a classifier trained with the ordinary cross-entropy loss tends to favour the majority class. Common remedies are re-weighting the loss in inverse proportion to class frequency, over-sampling the minority class, and loss functions such as the focal loss [16] that down-weight easy examples.

Imbalance also changes how a model should be evaluated. Accuracy is almost meaningless when 96% of images are benign. Saito and Rehmsmeier [17] showed that the precision–recall curve is more informative than the ROC curve on imbalanced data, because it reveals how many false alarms accompany each true detection. A decision threshold must also be chosen: Youden's J statistic [18], the threshold that maximises sensitivity + specificity − 1, is a widely used, prevalence-independent choice. Finally, with only a few dozen positive cases, proportions such as sensitivity are very uncertain, and confidence intervals such as the Wilson score interval [19] are needed to report them honestly. Cross-validation [30] makes the most of a small dataset by letting every image serve once as test data.

2.5. Explainability and Shortcut Learning

A clinical model must not only be accurate but should also be right for the right reasons. Gradient-weighted Class Activation Mapping (Grad-CAM) [20] produces a heat map showing which image regions most influenced a CNN's prediction, and it has become a standard tool for checking medical models. Such checks matter because CNNs are known to exploit shortcuts. Zech et al. [21] found that a pneumonia detector on chest X-rays partly relied on hospital-specific markers rather than lung findings, and Geirhos et al. [22] described shortcut learning as a general failure mode of deep networks. On ultrasound images, calipers, measurement tables and machine text are obvious candidates for such shortcuts, which is why this project removes them during preprocessing and checks the model with Grad-CAM.

2.6. Limitations of Existing Approaches

The literature shows that deep learning can classify ovarian masses well when large, carefully curated datasets are available [5], [6]. Several gaps remain, and this project is designed around them:

- **Data availability:** strong results come from large private datasets; public data such as OTU_2D is small and has very few malignant examples, so results on it are fragile.
- **Leakage and optimistic evaluation:** without patient identifiers, near-duplicate frames can leak between training and test sets, and thresholds tuned on test data inflate the reported performance.
- **Inadequate metrics:** accuracy alone hides the behaviour of a model on the rare malignant class; sensitivity, specificity and precision with confidence intervals are needed.
- **Shortcut learning:** on-screen annotations can be learned in place of anatomy, and few studies check for this explicitly.
- **Reproducibility:** many studies do not show whether their result can be reproduced by re-training.

This project addresses these gaps by building a clean, leakage-aware and fully reproducible baseline, evaluated with grouped cross-validation, validation-only model selection, class-sensitive metrics with confidence intervals, and Grad-CAM inspection.

III. SYSTEM ANALYSIS AND METHODOLOGY

3.1. System Architecture

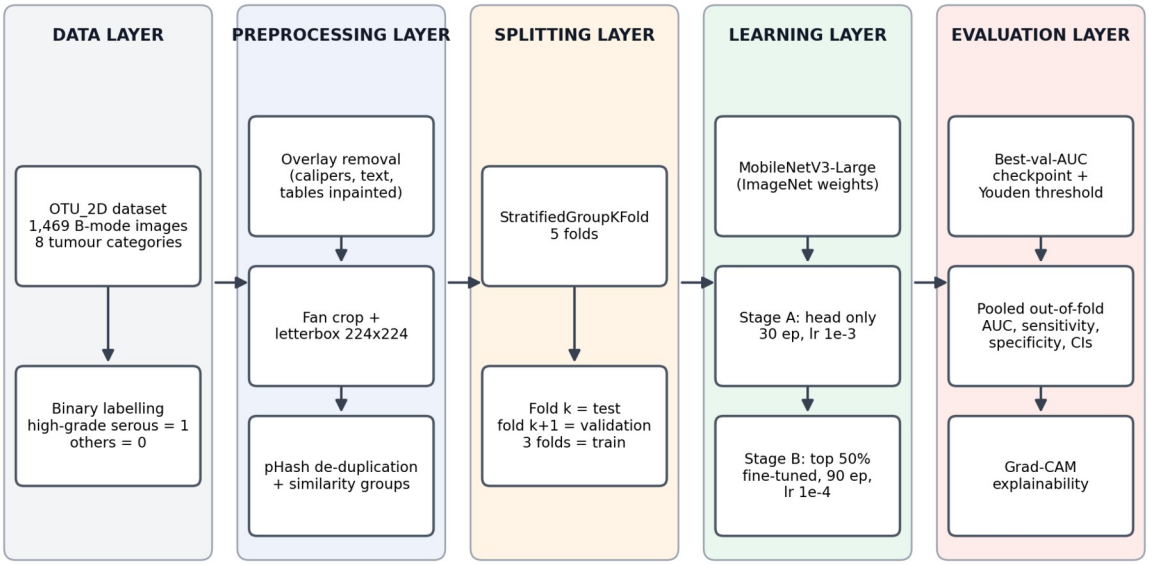


Fig. 1. System architecture of the E0 ovarian ultrasound classification pipeline

The system is a staged pipeline that turns raw ultrasound frames into a calibrated malignancy decision and an honest estimate of how well that decision performs. As shown in Fig. 1, it consists of five layers: data, preprocessing, splitting, learning and evaluation. Each layer reads the output of the previous one from disk and writes its own results, so any stage can be re-run or inspected on its own.

The **data layer** holds the 1,469 OTU_2D images and maps the eight diagnostic categories to a binary label, with high-grade serous carcinoma as the positive (malignant) class. The **preprocessing layer** removes on-screen calipers, measurement tables and text, crops each frame to the ultrasound fan, letterboxes it to 224 × 224 pixels and uses perceptual hashing to remove near-duplicate frames and group similar ones. The **splitting layer** assigns every image to one of five folds with a grouped, stratified procedure, and defines for every run which fold is used for training, validation and testing.

The **learning layer** fine-tunes an ImageNet-pretrained MobileNetV3-Large network in two stages, first training only the new classification head and then fine-tuning the upper half of the backbone. The **evaluation layer** keeps the checkpoint with the best validation AUC, fixes a decision threshold on the validation fold, evaluates the

untouched test fold, pools the five test folds into one overall result with confidence intervals, and produces Grad-CAM heat maps to show what the network looks at.

3.2. Technology Stack

The system is written entirely in Python and relies on widely used open-source libraries for deep learning, image processing and evaluation, as summarised in Table 1. Training ran on a consumer NVIDIA GPU inside the Windows Subsystem for Linux (WSL 2), which gives a full Linux CUDA environment on a Windows workstation.

Table 1 — Technology Stack Summary

Component	Tools	Purpose
Programming language	Python 3.12	Core development environment
Deep-learning framework	TensorFlow 2.20 / Keras	Model definition, training, checkpointing
Pretrained backbone	Keras Applications MobileNetV3-Large	ImageNet-initialised feature extractor
Image processing	OpenCV, Pillow, imagehash	Overlay removal, cropping, resizing, perceptual hashing
Data handling	NumPy, Pandas	Arrays, fold tables, logs and prediction files
Evaluation	scikit-learn, SciPy	ROC/PR metrics, thresholds, grouped folds, clustering
Visualisation	Matplotlib	Training curves, ROC curves, confusion matrices, Grad-CAM
Execution environment	WSL 2 (Ubuntu 24.04), CUDA, NVIDIA GTX 1650	GPU-accelerated training

3.2.1. Python Environment

Python was chosen because nearly every modern deep-learning and image-analysis library is available for it, and because it allows the whole pipeline, from cleaning to plotting, to be written in one language. All dependencies are installed in a dedicated virtual environment inside WSL, which isolates the project from system packages and keeps library versions fixed. The training script records its full argument list and the GPU name with every result, so each output file can be traced back to the exact configuration that produced it.

3.2.2. TensorFlow and Keras

TensorFlow with its high-level Keras API is used to build and train the network. Keras Applications provides MobileNetV3-Large with ImageNet weights and an in-built input rescaling layer, so raw 0–255 pixel values can be passed straight to the model. The tf.data API builds the input pipeline: images are decoded, cached in memory, shuffled, batched, augmented and prefetched, which keeps the GPU busy and makes an epoch take only a few seconds. Keras callbacks are used for checkpointing on validation AUC, for restoring the best weights at the end of each stage, and for a custom per-epoch logger.

3.2.3. Image Processing Libraries

OpenCV performs the image cleaning: detecting coloured overlay pixels, finding connected components, inpainting small annotations, locating the ultrasound fan as the largest bright region and cropping to it. Pillow reads and writes PNG files, and the imagehash library computes 64-bit perceptual hashes (pHash) that are used to measure how visually similar two frames are. SciPy's hierarchical clustering turns the pairwise hash distances into similarity groups.

3.2.4. Evaluation and Visualisation Libraries

scikit-learn provides the evaluation functions (ROC curve, ROC-AUC, average precision) and the StratifiedGroupKFold splitter used to build the folds. Pandas stores the fold assignment, the per-epoch training logs and the per-image predictions as CSV files, and JSON files store the final metrics. Matplotlib draws all training curves, ROC curves, confusion matrices and Grad-CAM overlays, and the plots are regenerated after every epoch so training can be watched live.

3.3. Dataset Description

The project uses OTU_2D, the two-dimensional B-mode subset of the MMOTU dataset [4], collected at Beijing Shijitan Hospital. It contains 1,469 still transvaginal ultrasound images in eight categories, together with lesion segmentation masks. Image sizes vary from 302 to 1,135 pixels wide and 266 to 794 pixels high. The dataset does not include patient identifiers. For this project only high-grade serous carcinoma is treated as malignant; the other seven categories are benign lesions or normal ovaries. Table 2 lists the number of images per category before and after cleaning, and Fig. 2 shows one cleaned example of each.

Table 2 — Summary of the Dataset Used

Category	Label	Raw	Dropped	Kept
Chocolate cyst (endometrioma)	Benign	336	30	306
Teratoma	Benign	336	32	304
Normal ovary	Benign	267	24	243
Serous cystadenoma	Benign	219	20	199
Mucinous cystadenoma	Benign	104	8	96
Theca cell tumour	Benign	88	6	82
Simple cyst	Benign	66	3	63
High-grade serous carcinoma	**Malignant**	**53**	**3**	**50**
Total		**1,469**	**126**	**1,343**

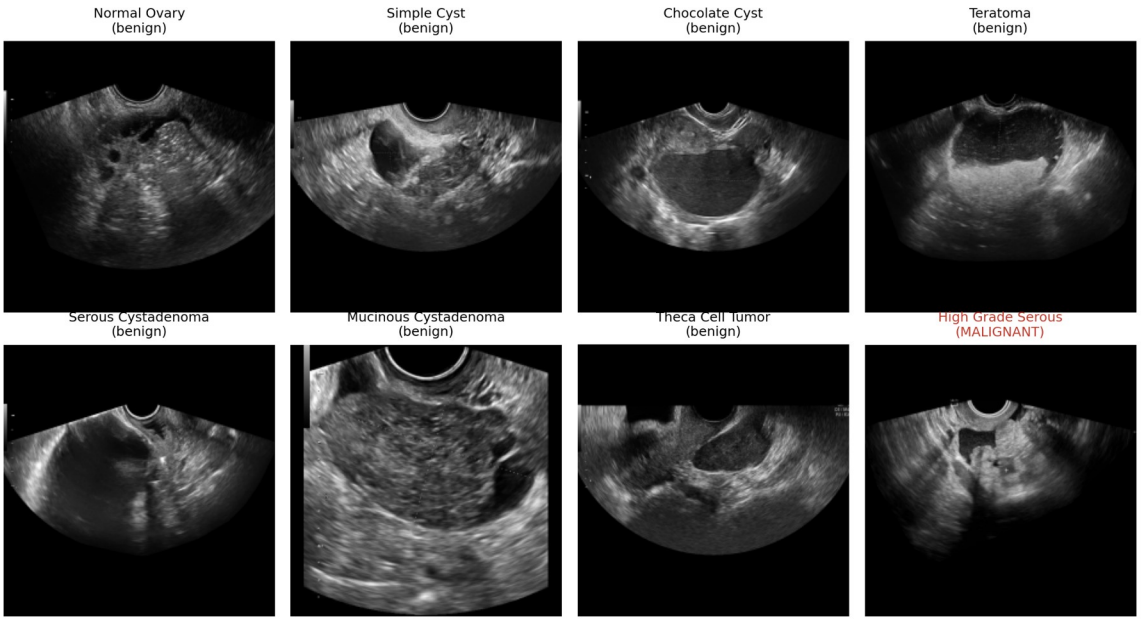


Fig. 2. One cleaned 224 × 224 example from each of the eight categories

Of the 126 dropped images, 110 were near-duplicate frames and 16 were near-duplicates whose label disagreed with their twin, which suggests a labelling error in one of the two. The cleaned set has **1,293 benign and 50 malignant images**, a positive rate of only 3.7% (Fig. 3). The 50 malignant images fall into about 46 visually distinct groups, so the number of truly independent malignant cases is smaller than 50, and the number of distinct patients may be smaller still.

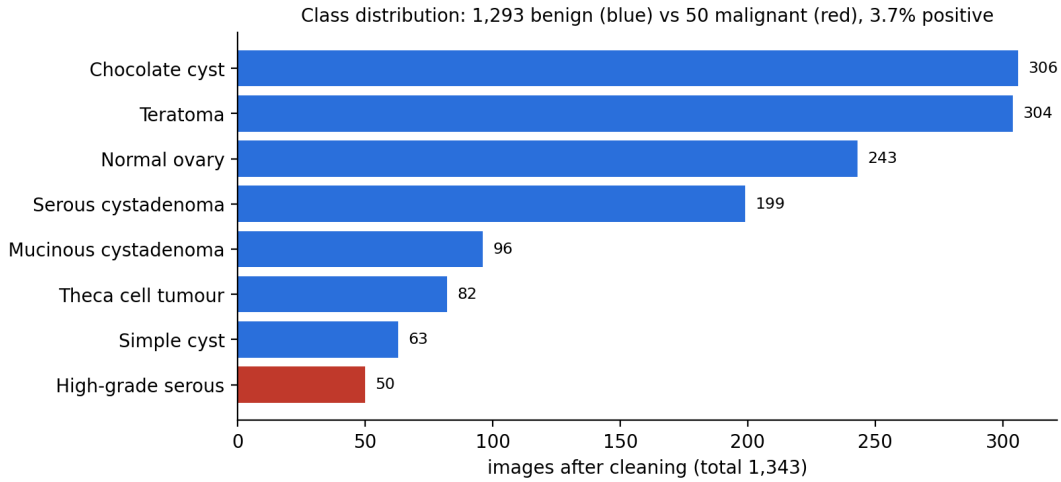


Fig. 3. Class distribution of the cleaned dataset

3.4. Model Selection

The choice of backbone network was driven by three requirements: the network must be available with reliable ImageNet weights, it must be small enough not to overfit 1,300 training images too badly, and it must train quickly on a 4 GB consumer GPU so that the full five-fold experiment can be repeated. Table 3 compares the candidate architectures using the approximate figures reported in their original papers.

Table 3 — Comparison of Candidate Backbone Networks (ImageNet, 224 × 224 input)

Network	Parameters	Multiply-adds	ImageNet top-1	Remark
ResNet-50 [11]	25.6 M	≈ 4.1 B	≈ 76%	Large; high overfitting risk on 1.3k images
EfficientNet-B0 [15]	5.3 M	≈ 0.39 B	≈ 77%	Accurate but slower to fine-tune
MobileNetV2 [13]	3.4 M	≈ 0.30 B	≈ 72%	Light; lower accuracy
MobileNetV3-Large [14]	**5.4 M**	**≈ 0.22 B**	**≈ 75%**	**Selected for E0**
MobileNetV3-Small [14]	2.5 M	≈ 0.06 B	≈ 67%	Lightest; candidate for future work

MobileNetV3-Large gives the best balance. Its ImageNet accuracy is close to ResNet-50 while it needs about one-fifth of the parameters and a twentieth of the computation. Without its 1,000-class ImageNet classifier the backbone has about **3.0 million parameters**, and the complete E0 model has 2,997,313 parameters. Its squeeze-and-excitation blocks [29] let it re-weight feature channels, which is useful for picking out subtle texture differences in ultrasound. A full five-fold run of E0 takes under an hour on the available GPU. MobileNetV3-Small, the lightest model in the table, is kept

in reserve: once the baseline is established, training a lighter model of this kind is a natural next step (Section 6.3).

3.5. Methodology Overview

The overall methodology is shown in Fig. 4. It starts with acquiring the raw images and cleaning them, then builds the grouped folds once and stores them in a file so every experiment uses exactly the same split. For each of the five folds, the model is trained with class weights and augmentation in two stages, the best checkpoint and the decision threshold are chosen on the validation fold, and the predictions on the test fold are saved. After all five folds have run, the test predictions are pooled and summarised. Finally, a deployable model is trained once on folds 0–3, with fold 4 used for checkpoint and threshold selection.

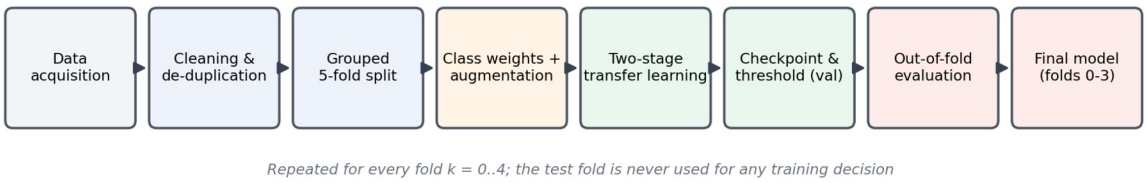


Fig. 4. Methodology overview of the E0 training and evaluation procedure

Two principles guide every step. The first is that **the test fold is never used for any decision**: checkpoint selection, restoration of the best weights and the threshold all use the validation fold, and test metrics are logged for monitoring only. The second is **reproducibility**: the folds are fixed in a CSV file, every random seed is set, and the complete configuration is saved with each result.

IV. SYSTEM DESIGN AND IMPLEMENTATION

4.1. Overall System Design

The system is built as a set of small, single-purpose programs that communicate through files. The cleaning and splitting stages run once and write a processed image folder and a fold-assignment table (cv_folds.csv). The training program, train_e0.py, runs one fold at a time in its own process, which keeps GPU memory clean between folds and means that a crash in one fold does not affect the others. A summary program, summarize_cv.py, reads the five sets of test predictions and produces the pooled result. A shell script, run_e0.sh, runs all five folds, the summary and the final model in sequence, writes everything to a log, and skips any fold that has already finished, so an interrupted run can be resumed.

Every training run writes the same set of artefacts: a per-epoch log (epoch_log.csv) with training, validation and test metrics, one plot per metric redrawn after every epoch, the kept checkpoint (best.keras), the last-epoch weights (final.keras), the chosen threshold (threshold.json), per-image predictions for the validation and test folds, and a metrics.json file that records the full configuration and the results.

4.2. Data Cleaning and Preprocessing

Raw OTU_2D frames contain much more than the ultrasound image itself. Inside the scan fan there are coloured calipers, dotted measurement lines and measurement tables; outside it there is machine text, scale bars and colour bars. If these were left in place, the network could learn, for example, that frames with many measurements are more likely to be malignant. The cleaning stage therefore applies the following steps to every frame:

- **Overlay detection:** a pixel is treated as coloured if the spread between its largest and smallest RGB value exceeds 40. Coloured connected components smaller than 0.4% of the frame are marked as annotations; larger coloured regions, which are colour-Doppler flow, are left alone.
- **Inpainting:** the marked annotation pixels are filled in from surrounding tissue with the Telea inpainting algorithm, and the frame is converted to grayscale.

- **Fan isolation:** the largest bright connected region is taken as the scan fan, filled using its convex hull; everything outside it is set to black and the frame is cropped to the fan's bounding box.
- **Letterboxing:** the cropped frame is scaled to fit inside 224 × 224 pixels while keeping its aspect ratio, and padded with black, so anatomy is never stretched. The single gray channel is repeated three times to match the network's RGB input.

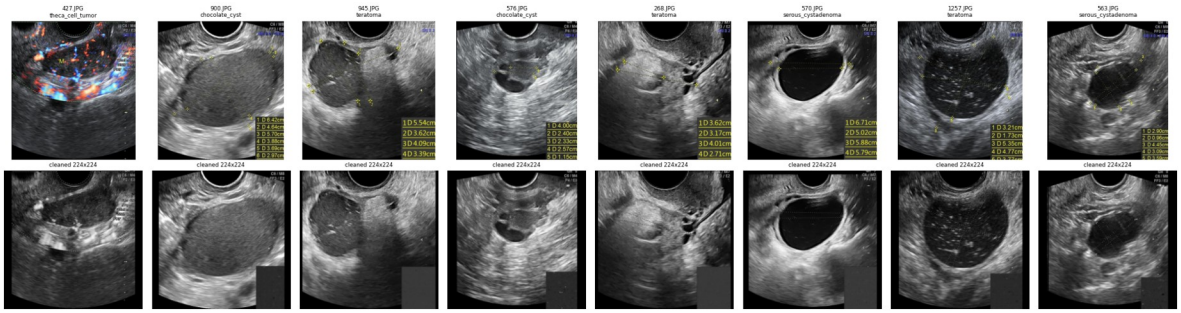


Fig. 5. Raw OTU_2D frames (top) and the corresponding cleaned 224 × 224 images (bottom)

Fig. 5 shows the effect of cleaning on eight frames. Calipers and text are removed and the fan fills the image. Some residual marks remain: where a large measurement table was inpainted, a uniform grey rectangle is left in 139 images, and a few faint caliper traces survive. These residues are discussed again in the Grad-CAM analysis of Chapter V.

Near-duplicate removal. For each cleaned frame a 64-bit perceptual hash is computed. Two frames whose hashes differ in at most 4 bits are considered near-identical and only one is kept; when the two copies carried different labels, both were dropped. This removed 126 images and left 1,343.

4.3. Leakage-aware Cross-validation Design

OTU_2D has no patient identifiers, so several frames of the same lesion may exist even after de-duplication. To approximate patient-level separation, frames are clustered with complete-linkage hierarchical clustering on their pHash distances: every pair of frames inside a group differs by at most 10 bits. This produces 1,117 groups. Complete linkage was chosen over simple chaining because chaining linked dissimilar frames into one very large group, which made balanced folds impossible.

The groups are then divided into five folds with scikit-learn's StratifiedGroupKFold, stratified on the eight original categories so each fold has a similar mix of lesion types. Fifty random seeds were tried and the most balanced assignment, measured by how evenly the images and the malignant images are spread, was kept and saved to cv_folds.csv. It was never re-randomised. A check over the saved file confirms that **none of the 1,117 groups appears in more than one fold**.

Each of the five runs uses one fold for testing, the next fold for validation and the remaining three for training, as shown in Fig. 6. Across the five runs, every image, and therefore every one of the 50 malignant images, is tested exactly once by a model that did not see it during training. Table 4 gives the composition of each fold.

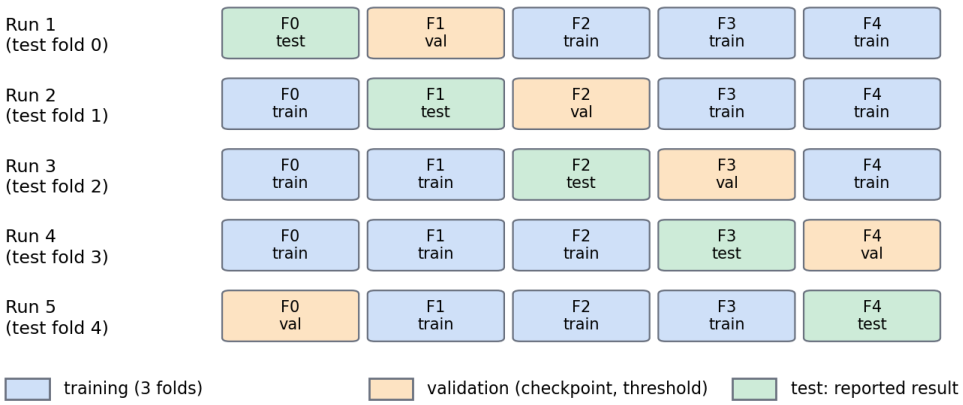


Fig. 6. Rotation of training, validation and test folds across the five cross-validation runs

Table 4 — Cross-validation Fold Composition

Test fold	Test images (malignant)	Validation fold	Training images (malignant)	Malignant class weight
0	266 (10)	1	812 (31)	13.10
1	265 (9)	2	808 (29)	13.93
2	270 (12)	3	806 (29)	13.90
3	267 (9)	4	801 (31)	12.92
4	275 (10)	0	802 (30)	13.37

4.4. Input Pipeline and Data Augmentation

Images are loaded with tf.data, decoded from PNG, cached in memory and served in batches of 32. The training stream is shuffled every epoch and repeated, and each batch is augmented on the fly. The validation and test streams are never

augmented. The E0 augmentation is intentionally mild, so that it does not create unrealistic ultrasound images:

- random horizontal flip;
- random rotation of up to ±15°, with the exposed corners filled with black;
- random brightness change of up to ±10% of the pixel range;
- random contrast change of up to ±10%.

After augmentation, pixel values are clipped back to the range 0–255. The network's own input layer then rescales them as MobileNetV3 expects.

4.5. Handling Class Imbalance

With about 26 benign images for every malignant one, an unweighted loss would be dominated by the benign class. E0 uses inverse-frequency class weights, computed separately for each training set as in Eq. (4.1), where N is the number of training images and N_c the number of images in class c:

$$w_c = \frac{N}{2 \cdot N_c} \tag{4.1}$$

For the fold-0 training set (812 images, 31 malignant) this gives a weight of 0.52 for benign and 13.10 for malignant images, so each malignant image counts about 25 times as much as a benign one. The weights multiply the binary cross-entropy loss of each image, as in Eq. (4.2), where y_i is the true label and p_i the predicted probability:

$$L = -\frac{1}{N} \Sigma \, w_y \, [\, y \, \log \, p + (1 - y) \, \log(1 - p) \,] \tag{4.2}$$

4.6. Network Architecture

The E0 network (Fig. 7) takes a 224 × 224 × 3 image. The MobileNetV3-Large backbone, loaded with ImageNet weights and without its original classifier, turns the image into a 7 × 7 × 960 feature map. Global average pooling reduces it to a 960-dimensional vector, dropout with rate 0.3 [24] is applied for regularisation, and a single dense unit with a sigmoid activation outputs the probability that the image is malignant. The backbone is always called in inference mode, so its batch-normalisation statistics [25] stay fixed even when its weights are being fine-tuned; with so little data, updating these statistics would make training unstable.

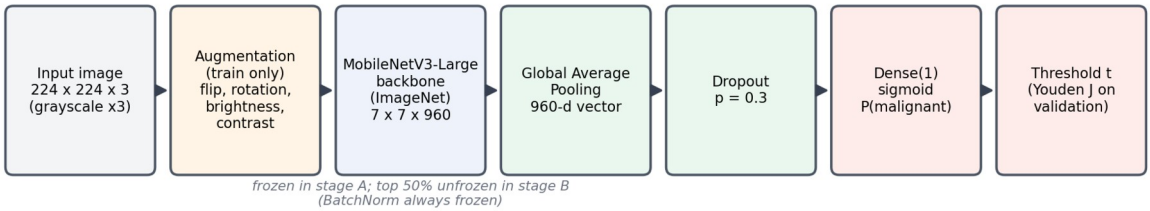


Fig. 7. E0 network architecture: MobileNetV3-Large backbone with a pooled, regularised sigmoid head

The core of the model definition in train_e0.py is shown below.

```
backbone = tf.keras.applications.MobileNetV3Large(
    weights="imagenet", include_top=False,
    input_shape=(224, 224, 3), include_preprocessing=True)
backbone.trainable = False
inp = tf.keras.Input((224, 224, 3), name="image")
x = backbone(inp, training=False)      # BatchNorm stays in inference mode
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.3)(x)
out = tf.keras.layers.Dense(1, activation="sigmoid", name="malignancy")(x)
model = tf.keras.Model(inp, out)
```

4.7. Two-stage Training Strategy

Training a newly initialised head together with a pretrained backbone from the start can damage the pretrained features, because the large, random gradients from the head flow back into the backbone. E0 therefore trains in two stages.

Stage A (head training) freezes the whole backbone and trains only the dense head for 30 epochs with the Adam optimiser [23] at a learning rate of 1×10^{-3}. At the end of stage A the weights with the lowest validation loss are restored. **Stage B (fine-tuning)** then unfreezes the top 50% of the backbone's layers, except the batch-normalisation layers, and continues for 90 epochs at a ten-times smaller learning rate of 1×10^{-4}. There is no early stopping: all 120 epochs are run, and the epoch with the best validation AUC across both stages is kept. Table 5 lists every setting.

Table 5 — E0 Training Configuration

Setting	Value
Backbone	MobileNetV3-Large, ImageNet weights, no top
Input	224 × 224, grayscale replicated to 3 channels
Head	Global average pooling → Dropout 0.3 → Dense(1, sigmoid)
Loss	Binary cross-entropy with inverse-frequency class weights
Optimiser	Adam

Setting	Value
Stage A	Backbone frozen, 30 epochs, lr 1×10^{-3}, restore best validation loss
Stage B	Top 50% of backbone unfrozen (BatchNorm frozen), 90 epochs, lr 1×10^{-4}
Batch size	32
Augmentation	Flip, rotation ±15°, brightness ±10%, contrast ±10%
Checkpoint selection	Best validation ROC-AUC over all 120 epochs
Decision threshold	Youden's J on the validation fold (lowest threshold among ties)
Random seed	42 + fold index

4.8. Checkpoint Selection and Decision Threshold

The sigmoid output is a probability, but a clinical decision needs a yes/no answer, so a threshold t must be chosen. Because the data is so imbalanced and the class weights shift the output probabilities, the conventional value of 0.5 is not appropriate. E0 chooses the threshold on the validation fold using Youden's J statistic [18], defined in Eq. (4.3), and takes the threshold at which J is largest:

$$J(t) = Sensitivity(t) + Specificity(t) - 1 \tag{4.3}$$

When several thresholds give the same maximum J, the lowest is taken, which favours sensitivity, since missing a cancer is worse than a false alarm. The kept checkpoint is reloaded, its predictions on the validation fold are used to fix t, and only then is the test fold scored with that fixed threshold. The test fold therefore plays no part in choosing either the weights or the threshold.

4.9. Evaluation Metrics

Writing TP, FN, TN and FP for the numbers of true positives, false negatives, true negatives and false positives, the main threshold-dependent metrics are:

$$Sensitivity = \frac{TP}{TP + FN}, \quad Specificity = \frac{TN}{TN + FP} \tag{4.4}$$

$$PPV = \frac{TP}{TP + FP}, \quad NPV = \frac{TN}{TN + FN} \tag{4.5}$$

Sensitivity (recall) is the fraction of malignant images that are detected and is the most important metric for this application. Specificity is the fraction of benign images correctly cleared. The positive predictive value (PPV, or precision) tells how often an alarm is really cancer, which depends strongly on how rare cancer is. Two

threshold-free metrics summarise ranking quality: the **ROC-AUC**, the probability that a randomly chosen malignant image receives a higher score than a randomly chosen benign one, and the **PR-AUC** (average precision), which is more sensitive to false alarms under imbalance [17].

The headline numbers are **pooled out-of-fold** metrics: the 1,343 test predictions from the five folds, each made at its own fold's threshold, are combined and evaluated together. With only 50 malignant images, 95% confidence intervals for sensitivity and specificity are computed with the Wilson score interval [19], which behaves well for small counts and proportions near 0 or 1.

4.10. Implementation and Software Organisation

The implementation is organised so that the complete experiment can be reproduced with a single command. The main components are:

- **train_e0.py:** trains one fold (--mode cv --fold k) or the final model (--mode final). It refuses to run if no GPU is visible, checks that every image exists before training, logs every epoch, and writes the checkpoint, threshold, predictions and metrics.
- **summarize_cv.py:** pools the five test-fold prediction files, computes the pooled metrics with Wilson intervals, compares them with an earlier run, and draws the pooled ROC curve and all-fold training curves.
- **run_e0.sh:** runs folds 0–4, the summary and the final model in sequence inside WSL, appends all output to run.log and stops if any fold fails.
- **cv_folds.csv:** the fixed fold table, with the file name, category, label, similarity group and fold of every image.

A custom Keras callback writes one row per epoch to epoch_log.csv, evaluates the test fold for monitoring, saves the latest weights so a run can be inspected at any time, and redraws six metric plots (loss, accuracy, ROC-AUC, PR-AUC, recall and precision). A second callback remembers the best weights during a stage and restores them at the end of that stage. A finished run is detected from its metrics.json file and skipped on re-run unless --force is given.

V. RESULTS AND DISCUSSION

5.1. Testing Environment

All experiments were run on a Windows 11 laptop with an NVIDIA GeForce GTX 1650 GPU (4 GB), using Ubuntu 24.04 inside the Windows Subsystem for Linux, which gives TensorFlow direct access to the GPU through CUDA. Before each run, the training script confirms that a GPU is visible and that a test matrix multiplication actually executes on it, and it refuses to train on the CPU. Table 6 summarises the environment.

Table 6 — Testing Environment

Item	Details
Host operating system	Windows 11 Home
Training environment	WSL 2, Ubuntu 24.04.3 LTS, Python 3.12.3 virtual environment
GPU	NVIDIA GeForce GTX 1650, 4 GB (driver 572.16)
Main libraries	TensorFlow 2.20.0, scikit-learn 1.9.1, NumPy 2.5.3, Pandas 3.0.6
Average epoch time	≈ 2.9 s (training + validation + test monitoring)
Time per fold (120 epochs)	9.8 – 10.4 min
Complete run (5 folds + summary + final model)	≈ 64 min

5.2. Test Cases and Evaluation Set-up

Two kinds of testing were carried out. The first checks that the software pipeline behaves correctly and that the evaluation cannot be contaminated; the second measures the model's classification performance. Table 7 lists the pipeline test cases and their outcomes, all of which were verified from the saved output files and the run log.

Table 7 — Pipeline Test Cases

ID	Test case	Expected result	Outcome
TC1	GPU check before every run	Test operation executes on GPU:0, otherwise training stops	Pass (6/6 runs on GPU)
TC2	Image availability check	All images of the train/val/test folds exist before training	Pass (1,343/1,343 found)
TC3	Fold leakage check	No similarity group appears in two folds	Pass (0 of 1,117 groups)
TC4	Coverage of malignant images	Each of the 50 malignant images tested exactly once	Pass (50/50, 1,343 predictions)

ID	Test case	Expected result	Outcome
TC5	Per-fold class weights	Malignant weight ≈ 13 computed from each training set	Pass (12.92 – 13.93)
TC6	Validation-only decisions	Checkpoint and threshold chosen on validation fold	Pass (recorded in metrics.json)
TC7	Full schedule	All 120 epochs run in every fold (no early stop)	Pass (5 × 120 epochs logged)
TC8	Reproducibility	Re-run lands close to an earlier run of the same configuration	Pass (see Section 5.6)

For the performance evaluation, the protocol of Chapter IV was used throughout: five runs, each with one test fold, one validation fold and three training folds; best-validation-AUC checkpoint; Youden threshold on validation; and test-fold predictions pooled across the five runs. Test-set metrics were logged at every epoch purely for monitoring and were never used to make a decision.

5.3. Training Behaviour

Fig. 8 shows the loss and ROC-AUC curves of fold 0. In stage A (epochs 1–30), when only the head is trained, the training and validation curves move together slowly and the validation AUC levels off around 0.56. As soon as fine-tuning starts in stage B, the training AUC rises steeply and reaches 1.000 within about 15 epochs, while the training loss falls almost to zero. The validation and test AUC also improve sharply at first, but the validation loss starts to rise after about epoch 45, a clear sign of overfitting. The kept checkpoint (epoch 53, red dotted line) lies at the peak of the validation AUC, before most of the overfitting sets in.

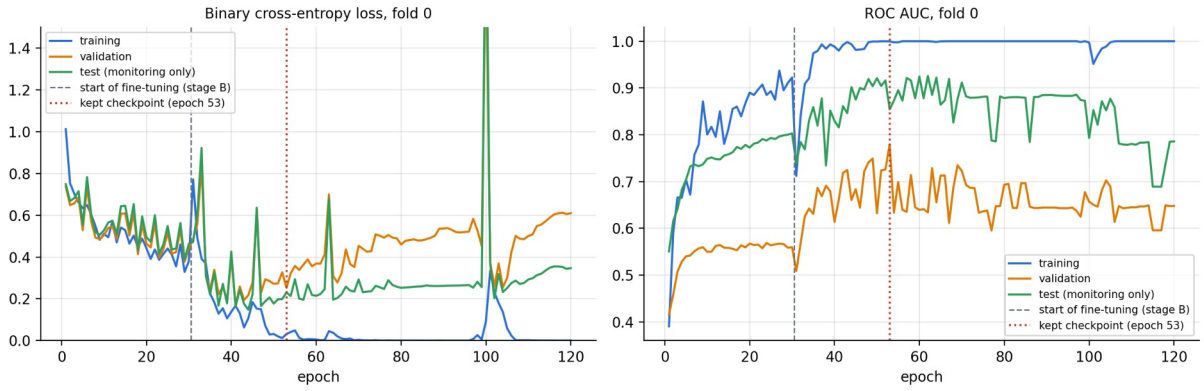


Fig. 8. Loss and ROC-AUC curves for fold 0 (training, validation and test)

Fig. 9 shows that the same pattern appears in all five folds. Every fold reaches a training AUC of 1.000, which means the network can perfectly memorise its roughly

800 training images, while the validation AUC peaks between 0.79 and 0.92 and then drifts down or fluctuates. The kept epochs range from 34 to 94, all within stage B. Fine-tuning is therefore essential for good performance, but with so few malignant examples the model overfits quickly, and selecting the checkpoint on a separate validation fold matters a great deal.

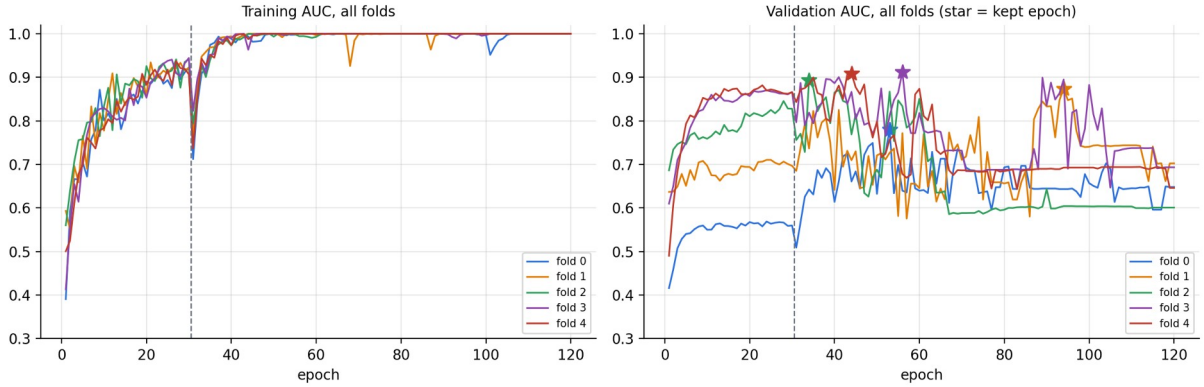


Fig. 9. Training and validation ROC-AUC for all five folds

5.4. Per-fold Performance

Table 8 and Fig. 10 report the results on each test fold, using the threshold chosen on that run's validation fold.

Table 8 — Per-fold Test Results of E0

Fold	Kept epoch	Threshold	ROC-AUC	PR-AUC	Malignant caught	False alarms	Specificity	Accuracy
0	53	0.007	0.867	0.332	8 / 10	28 / 256	0.891	0.887
1	94	0.036	0.765	0.393	5 / 9	24 / 256	0.906	0.894
2	34	0.396	0.823	0.218	9 / 12	63 / 258	0.756	0.756
3	56	0.017	0.836	0.436	5 / 9	19 / 258	0.926	0.914
4	44	0.084	0.916	0.345	7 / 10	22 / 265	0.917	0.909
Mean			**0.841 ± 0.056**	**0.345**			**0.879**	**0.872**

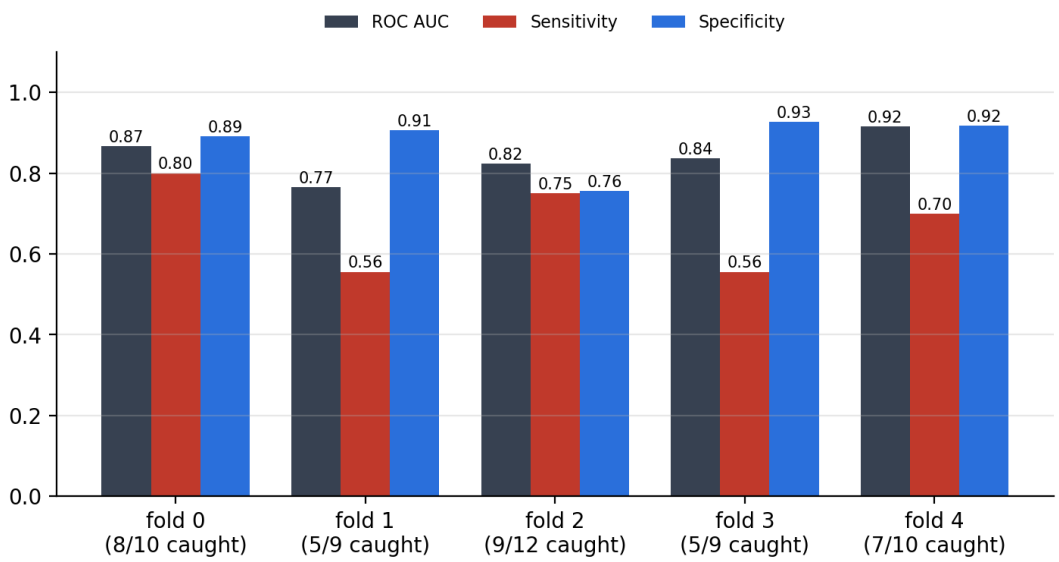


Fig. 10. ROC-AUC, sensitivity and specificity on each test fold

The fold AUC ranges from 0.765 to 0.916, a spread of about 0.15 produced by the same model and the same training recipe. Much of this variation reflects which particular malignant images happen to fall in each test fold: with only 9 to 12 malignant images per fold, a single image moves the fold's sensitivity by 8 to 11 percentage points. The chosen thresholds also vary widely, from 0.007 to 0.396. The class weights push the output probabilities around differently in each run, so a fixed threshold such as 0.5 would have behaved very differently from fold to fold; choosing the threshold on validation data handles this automatically. Fold 2, with the highest threshold, traded more false alarms for higher sensitivity.

5.5. Pooled Cross-validation Performance

Pooling the 1,343 out-of-fold test predictions gives the headline result of the project, shown in Table 9.

Table 9 — Pooled 5-fold Out-of-fold Results of E0

Metric	Value	95% CI / remark
ROC-AUC (pooled)	**0.819**	fold mean 0.841 ± 0.056
PR-AUC (pooled)	0.279	chance level = prevalence = 0.037
Sensitivity (malignant caught)	**34 / 50 = 68.0%**	54.2% – 79.2% (Wilson)
Specificity (benign cleared)	**1,137 / 1,293 = 87.9%**	86.0% – 89.6% (Wilson)
False alarms	156 / 1,293 benign	12.1% of benign images flagged
Positive predictive value	34 / 190 = 17.9%	about 1 in 6 alarms is cancer

Metric	Value	95% CI / remark
Negative predictive value	1,137 / 1,153 = 98.6%	a ‘benign’ result is right 98.6% of the time
Balanced accuracy	78.0%	(sensitivity + specificity) / 2
Accuracy	87.2%	dominated by the benign class
F1-score (malignant)	0.283	harmonic mean of PPV and sensitivity

Fig. 11 shows the pooled ROC curve together with the five per-fold curves, and Fig. 12 the pooled confusion matrix. The pooled AUC of 0.819 means that in about 82% of malignant–benign pairs the malignant image receives the higher score. At the per-fold Youden thresholds, the model catches 34 of the 50 malignant images while clearing 87.9% of benign ones. The PR-AUC of 0.279 is low in absolute terms but about 7.5 times the chance level of 0.037, which reflects how hard it is to keep precision high when cancer is this rare.

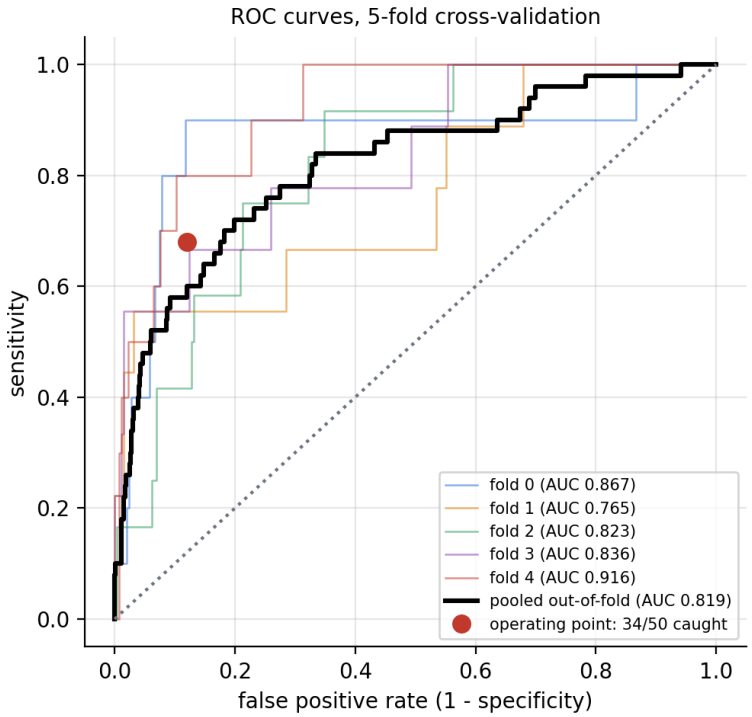


Fig. 11. Pooled and per-fold ROC curves with the operating point at the Youden thresholds

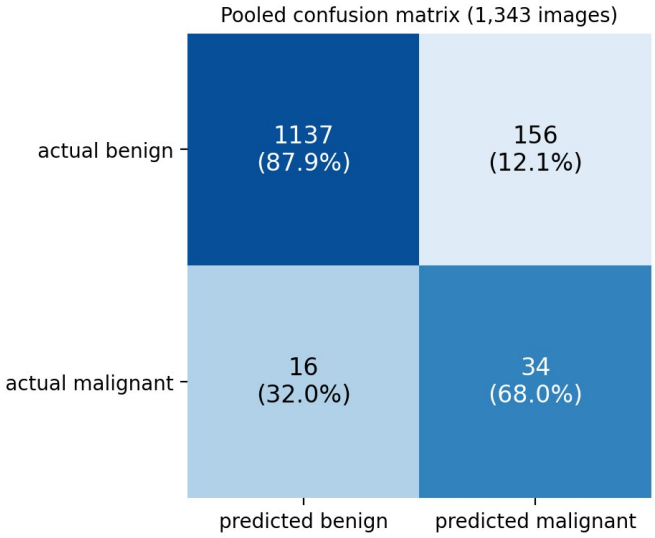


Fig. 12. Pooled confusion matrix over all 1,343 out-of-fold predictions

These numbers should be read with their uncertainty in mind. The 95% confidence interval for sensitivity spans 54% to 79%: with only 50 positives the true sensitivity of this model could plausibly be anywhere in that range. Specificity is measured much more precisely because there are 1,293 benign images. The negative predictive value of 98.6% looks reassuring, but it mostly reflects how rare cancer is in this dataset; 16 malignant images were still missed. The model is therefore better suited to ranking images by suspicion, as a second reader, than to making stand-alone decisions.

5.6. Reproducibility of the Baseline

GPU training is not bit-for-bit deterministic, so it is important to show that the result does not depend on a lucky run. The complete five-fold E0 experiment was run twice with exactly the same code, folds and settings. Table 10 compares the two runs.

Table 10 — Reproducibility of E0 Across Two Independent Runs

Metric	Earlier run	This run (reported)
Pooled ROC-AUC	0.823	0.819
Fold AUC mean ± SD	0.825 ± 0.036	0.841 ± 0.056
Malignant caught	35 / 50	34 / 50
Specificity	0.779 (286 false alarms)	0.879 (156 false alarms)
Positive predictive value	10.9%	17.9%

The ranking quality is essentially identical (AUC 0.823 versus 0.819), and the number of malignant images caught differs by one. The specificity differs more,

because the Youden threshold in each fold is chosen on only 9 to 12 malignant validation images and is therefore itself somewhat unstable; in this run the thresholds happened to sit at points that gave far fewer false alarms for almost the same sensitivity. The comparison shows that the E0 pipeline is reproducible in its ranking ability, and that the operating point is the least stable part of the system, a direct consequence of the small number of malignant examples.

5.7. Final Model

After cross-validation, a single final model was trained on folds 0–3 (1,068 images, 40 malignant), with fold 4 (275 images, 10 malignant) used to select the checkpoint and the threshold. Its training curves (Fig. 13) follow the same pattern as the cross-validation folds. Table 11 summarises the final model.

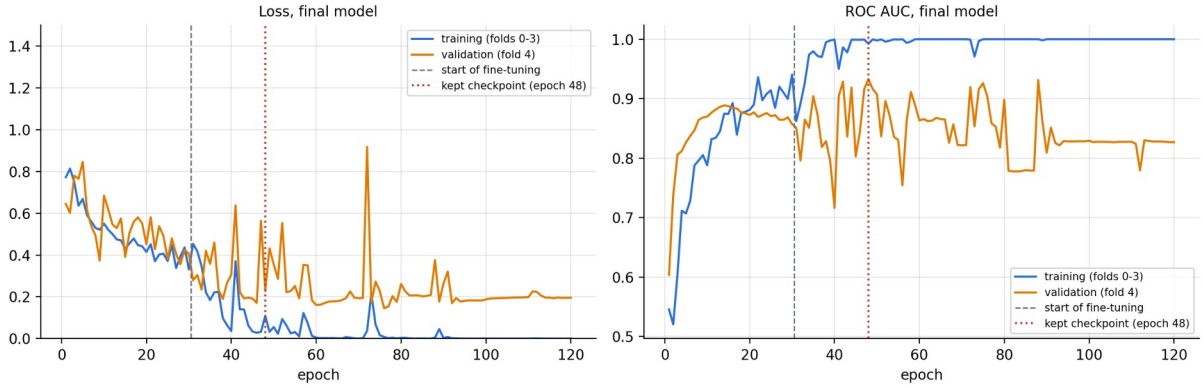


Fig. 13. Loss and ROC-AUC curves of the final model trained on folds 0–3

Table 11 — Final E0 Model Summary

Item	Value
Training data	Folds 0–3: 1,068 images (40 malignant)
Validation data	Fold 4: 275 images (10 malignant)
Kept checkpoint	Stage B, epoch 18 (overall epoch 48)
Decision threshold	0.110 (Youden J on fold 4)
Validation ROC-AUC	0.932 (optimistic: same data chose checkpoint and threshold)
Validation sensitivity / specificity	10 / 10 caught, specificity 0.834
Parameters / saved model size	2,997,313 parameters, 35.2 MB (.keras)
Training time	9.5 min

The final model has no separate test set, so its validation figures are optimistic and must not be quoted as its expected performance. The expected performance of this model on new images from the same source is given by the cross-validation estimate of

Section 5.5: an AUC of about 0.82 and a sensitivity of about 68% at a specificity of about 88%.

5.8. Model Interpretability with Grad-CAM

To understand what the network bases its decisions on, Grad-CAM [20] heat maps were computed for out-of-fold predictions of the E0 configuration and compared with the lesion outlines provided with the dataset. Fig. 14 shows typical true positives, false negatives, false positives and true negatives.

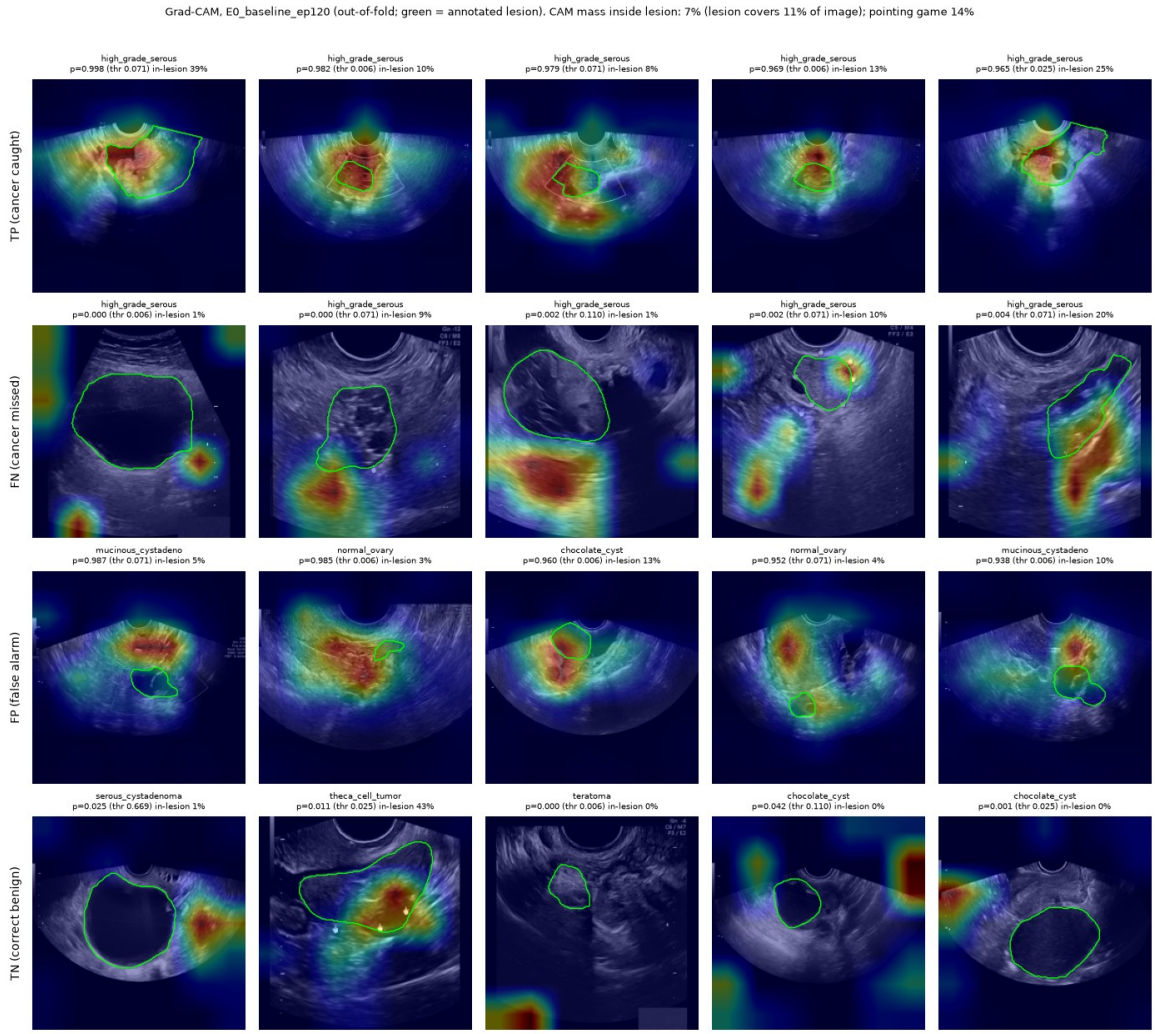


Fig. 14. Grad-CAM heat maps for E0 out-of-fold predictions (green outline = annotated lesion). Rows: true positives, false negatives, false positives, true negatives

For confidently detected cancers (top row), the heat map usually covers the solid, irregular part of the tumour, which is the region an expert would also examine. The misses (second row) are instructive: in several of them the network's attention lies outside the lesion, on surrounding tissue, the edge of the fan or a residual screen artefact,

and the predicted probability is close to zero. Many false alarms (third row) are benign lesions with solid or complex internal structure, such as mucinous cystadenomas and some chocolate cysts, which are genuinely difficult cases for human readers too.

Averaged over the dataset, only a small fraction of the Grad-CAM mass falls inside the annotated lesion. In some images the network attended to the uniform grey rectangle left where a measurement table had been inpainted. This is a warning sign of shortcut learning [21], [22]: with only 50 malignant examples, any artefact that happens to correlate with the label can be picked up. Removing these residues more completely, or masking the input to the lesion region, is an important direction for improving the model.

5.9. Discussion

The results support several conclusions about the E0 baseline:

- **Transfer learning works on this task.** A pretrained MobileNetV3-Large, fine-tuned on about 800 images per fold, separates high-grade serous carcinoma from benign ovarian lesions with a pooled AUC of 0.82, far above chance, using only single grayscale frames.
- **Fine-tuning is essential but overfits quickly.** Head-only training plateaued at a low validation AUC; unfreezing the upper backbone brought large gains, but training AUC reached 1.000 in every fold, and validation selection was necessary to stop at a good point.
- **The data, not the training recipe, limits performance.** The perfect training fit, the large fold-to-fold variation (AUC 0.77–0.92) and the wide confidence interval for sensitivity all point to the small number of malignant images (50 images, about 46 independent cases, one cancer subtype) as the main constraint.
- **The evaluation is trustworthy.** Duplicate removal, grouped folds and validation-only decisions mean that the reported 0.819 AUC and 34/50 sensitivity are fair estimates for new images from the same hospital, and a second independent run confirmed them.

VI. CONCLUSIONS AND SCOPE FOR FURTHER WORK

6.1. Conclusions

This project designed, implemented and evaluated E0, a deep transfer-learning baseline for detecting malignancy in two-dimensional ovarian ultrasound images. The system cleans the OTU_2D images of on-screen annotations, removes near-duplicate frames, builds grouped and stratified cross-validation folds that keep near-identical frames apart, and fine-tunes an ImageNet-pretrained MobileNetV3-Large network in two stages with class weighting and light augmentation. Every model-selection decision is made on validation data only.

Under this strict protocol, E0 achieved a pooled out-of-fold ROC-AUC of 0.819 and detected 34 of the 50 malignant images (sensitivity 68%, 95% CI 54–79%) at a specificity of 87.9%. A second, independent run of the same configuration reproduced the result (AUC 0.823, 35/50). The training curves, the fold-to-fold variation and the Grad-CAM analysis consistently show that the network has enough capacity to learn the task, and in fact memorises its training data, and that the small and narrow malignant class is the main factor limiting performance. The project therefore delivers a clean, reproducible and honestly evaluated reference baseline against which future models can be compared.

6.2. Limitations

- **Very few malignant examples.** Only 50 malignant images (about 46 independent cases) are available, so sensitivity can only be measured to within roughly ±12 percentage points, and one image changes a fold's sensitivity by about 10 points.
- **Only one cancer subtype.** All malignant images are high-grade serous carcinoma; borderline tumours and other carcinomas are absent, so the model has not learned "malignant" in general.
- **No patient identifiers.** Patient-level separation is approximated by image-similarity grouping; frames of the same patient that look different could still fall into different folds.

- **Single centre, still images.** All images come from one hospital, so performance on other scanners, operators and populations is unknown, and no video or Doppler information is used.
- **Residual artefacts.** Inpainted measurement tables leave grey boxes in 139 images and a few caliper traces remain; Grad-CAM shows the model sometimes attends to them.
- **Unstable operating point.** The decision threshold is chosen on 9–12 malignant validation images per fold, which makes the specificity at that threshold vary noticeably between runs.

6.3. Future Enhancements

The most immediate next step is to **train a lightweight model** that keeps the accuracy of E0 at a fraction of its size and computation. E0 already uses an efficient backbone, but MobileNetV3-Small has less than half the parameters and about a quarter of the computation of MobileNetV3-Large (Table 3), and a lighter network is also less able to memorise a small training set. The planned work is to fine-tune MobileNetV3-Small and other compact backbones on exactly the same folds and protocol so that their results are directly comparable with E0, and to use knowledge distillation [28], with E0 acting as the teacher, to transfer its knowledge into the smaller student network. A lighter model would train faster, make repeated cross-validation experiments cheaper, and give quicker predictions on ordinary computers.

Other directions for future work are:

- **More and more varied malignant data:** adding pathology-confirmed images from other centres and other cancer subtypes, and running a learning-curve experiment to confirm how much more data would help.
- **Stronger artefact removal:** blacking out measurement tables rather than inpainting them, and training on lesion-focused crops so the network cannot use screen artefacts as shortcuts.
- **Better imbalance handling:** focal loss [16], minority over-sampling and ultrasound-specific augmentation such as speckle noise and gain changes.
- **More stable thresholds:** choosing the operating point from pooled validation predictions or nested cross-validation, and calibrating the output probabilities.

- **Multi-class and multi-frame models:** predicting all eight lesion types, and combining several frames or video clips of the same lesion.
- **External validation:** testing the model on an independent dataset from another hospital before any clinical use is considered.

VII. REFERENCES

[1] H. Sung, J. Ferlay, R. L. Siegel, M. Laversanne, I. Soerjomataram, A. Jemal and F. Bray, "Global Cancer Statistics 2020: GLOBOCAN Estimates of Incidence and Mortality Worldwide for 36 Cancers in 185 Countries," CA: A Cancer Journal for Clinicians, vol. 71, no. 3, pp. 209–249, 2021.

[2] D. Timmerman et al., "Simple ultrasound-based rules for the diagnosis of ovarian cancer," Ultrasound in Obstetrics & Gynecology, vol. 31, no. 6, pp. 681–690, 2008.

[3] R. F. Andreotti et al., "O-RADS US Risk Stratification and Management System: A Consensus Guideline from the ACR Ovarian-Adnexal Reporting and Data System Committee," Radiology, vol. 294, no. 1, pp. 168–185, 2020.

[4] Q. Zhao, S. Lyu, W. Bai, L. Cai, B. Liu, M. Wu, X. Sang, M. Yang and L. Chen, "A Multi-Modality Ovarian Tumor Ultrasound Image Dataset for Unsupervised Cross-Domain Semantic Segmentation," arXiv preprint arXiv:2207.06799, 2022.

[5] F. Christiansen et al., "Ultrasound image analysis using deep neural networks for discriminating between benign and malignant ovarian tumors: comparison with expert subjective assessment," Ultrasound in Obstetrics & Gynecology, vol. 57, no. 1, pp. 155–163, 2021.

[6] Y. Gao et al., "Deep learning-enabled pelvic ultrasound images for accurate diagnosis of ovarian cancer in China: a retrospective, multicentre, diagnostic study," The Lancet Digital Health, vol. 4, no. 3, pp. e179–e187, 2022.

[7] G. Litjens et al., "A survey on deep learning in medical image analysis," Medical Image Analysis, vol. 42, pp. 60–88, 2017.

[8] A. Esteva et al., "Dermatologist-level classification of skin cancer with deep neural networks," Nature, vol. 542, pp. 115–118, 2017.

[9] M. Raghu, C. Zhang, J. Kleinberg and S. Bengio, "Transfusion: Understanding Transfer Learning for Medical Imaging," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[10] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248–255.

[11] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[12] A. G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” arXiv preprint arXiv:1704.04861, 2017.

[13] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510–4520.

[14] A. Howard et al., “Searching for MobileNetV3,” in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2019, pp. 1314–1324.

[15] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in Proc. Int. Conf. Machine Learning (ICML), 2019, pp. 6105–6114.

[16] T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, “Focal Loss for Dense Object Detection,” in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2017, pp. 2980 –2988.

[17] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” PLoS ONE, vol. 10, no. 3, e0118432, 2015.

[18] W. J. Youden, “Index for rating diagnostic tests,” Cancer, vol. 3, no. 1, pp. 32–35, 1950.

[19] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.

[20] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization,” in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2017, pp. 618–626.

[21] J. R. Zech, M. A. Badgeley, M. Liu, A. B. Costa, J. J. Titano and E. K. Oermann, “Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study,” PLoS Medicine, vol. 15, no. 11, e1002683, 2018.

[22] R. Geirhos et al., “Shortcut learning in deep neural networks,” Nature Machine Intelligence, vol. 2, pp. 665–673, 2020.

[23] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[24] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[25] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in Proc. Int. Conf. Machine Learning (ICML), 2015, pp. 448–456.

[26] M. Abadi et al., “TensorFlow: A System for Large-Scale Machine Learning,” in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

[27] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[28] G. Hinton, O. Vinyals and J. Dean, “Distilling the Knowledge in a Neural Network,” arXiv preprint arXiv:1503.02531, 2015.

[29] J. Hu, L. Shen and G. Sun, “Squeeze-and-Excitation Networks,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 7132–7141.

[30] R. Kohavi, “A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection,” in Proc. Int. Joint Conf. Artificial Intelligence (IJCAI), 1995, pp. 1137–1143.